

Homework 3

*Handed Out: 02/16/2026**Due: 03/02/2026*

In this assignment, you will practice classical bandit algorithms with simulated environments.

- Implement MAB algorithms including Explore then Commit, UCB, Thompson Sampling
- Implement linear bandit algorithms including LinUCB, LinTS
- Bonus: implement non-linear bandit algorithms.

Simulation environment. *hw3_startercode.zip* contains scripts for running simulation and example implementation of bandit algorithms. The script *Simulation.py* will generate action set “articles”, which is a set of Article objects (defined in *Articles.py*) storing id and feature vector of an article; and user set “users”, which is a set of User objects (defined in *Users.py*) storing id and unobservable linear parameter (that parameterizes the reward function) of a user. Hyperparameters:

- `testing_iterations`: total number of time steps to run.
- `NoiseScale`: standard deviation of the **Gaussian noise** in the reward
- `n_articles`: total number of articles in the articles set
- `n_users`: total number of users
- `poolArticleSize`: If it is set to `None`, in each time step, the action set contains all articles. Otherwise, randomly sample a size “`poolArticleSize`” subset of articles in each time step as the action set. This is the setting that many linear bandit adopts.
- `actionset`: Setting to “`basis_vector`” or “`random`”. “`basis_vector`” constrains the articles feature vectors to be basis vector like $e_0 = [1, 0, 0, \dots, 0]$, $e_1 = [0, 1, 0, \dots, 0]$. Therefore, feature vectors of the articles will be orthogonal, which means observation about reward of one article brings no information about reward of another article. “`random`” means using randomly sampled vectors from l2 unit ball.
- `context_dimension`: dimension of article feature vector and user linear parameter.

Task.

1. (60 points) Multi-armed Bandits. To simulate a K-armed bandit environment, set: `actionset = “basis_vector”`, `n_articles = context_dimension = K`. Then you can try different values of number of arms and standard deviation of Gaussian noise.

- ϵ -greedy: See `EpsilonGreedyMultiArmedBandit.py` under `lib` directory for example of how to implement the multi-armed bandit algorithm. Remember that in MAB environment the algorithm should not use `article.featureVector`, but only `article.id`.
 - (20 points) Explore then Commit: report the performance with different choices of exploration phase m (try at least 10 different values). Which m is the best? Why?
 - (20 points) UCB: write down the equation used for calculating UCB and report the performance.
 - (20 points) Thompson Sampling: write down the posterior distribution used for sampling and its prior, and report the performance.
2. (40 points) Linear Bandits. In linear bandits, each user has an unobservable linear parameter that parameterizes the linear reward function that takes the feature vector of the articles as input. In each time step, the learner observes the feature vectors of the action set, selects an arm and observes the reward associated with the selected arm. Therefore, the learner maintains sufficient statistics and constructs a linear regression estimator / updates posterior for the unknown linear parameter. To simulate a linear bandit environment, set: `actionset = "random"`. You may try different values of dimension of feature vector and linear parameter (`context_dimension`), `NoiseScale`, `n_articles`, `poolArticleSize`.
- (20 points) LinUCB: write down the equations used for parameter estimation and calculating UCB in report, and report the performance. You should report the details of every variable/hyperparameter such as α_t , λ , δ , etc.
 - (20 points) LinTS: write down the posterior distribution used for sampling and its prior in report, and report the performance. You should report the details of every variable/hyperparameter such as λ , δ , etc.
3. (Bonus, 15 points) Non-linear Bandits. Current environment is simulating linear rewards. Modify the environment to simulate non-linear function of rewards of your choice and implement a non-linear bandit algorithm to solve the new environment (e.g., generalized linear model, kernel bandits, or neural bandits). Report the reward function, the details of the non-linear bandit algorithm, and its performance. Compare the performance with any linear bandit model (LinUCB/LinTS).

What to submit. Submit a report in PDF using the homework template and a zip file of executable codes. Clearly list the procedure of your implemented algorithm: what are the inputs, and what are the steps in your report. Write down the commands used to generate the results in your report.

Besides lectures, you may also refer to [1] regarding the algorithms.

References

- [1] T. Lattimore and C. Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.